# AVR Digital IO and Matrix Keypads

Read the explanation first. Then build the simulation, complete the scanning function, test it, and answer the questions at the end.

## What you should learn

- Must understand: pin direction, output level, input pull-ups, active-low inputs, row and column scanning, and why a switch needs debounce.
- Should understand: how a key becomes a four-bit code, how bit masks preserve unrelated pins, and the difference between CPU timing and human-scale switch timing.
- Nice to know: two simultaneous keys can cause ambiguous readings; this exercise handles one key at a time.

## 1. How an AVR pin behaves

The ATmega328P groups digital pins into ports. Each port has three registers that work together. The x in DDRx, PORTx, and PINx is the port letter, such as B, C, or D.

| Register | What software controls or reads | Example |
|---|---|---|
| DDRx | Direction of each bit: 1 is output; 0 is input. | DDRD bit 4 = 0 → PD4 is input |
| PORTx | Output level if DDRx = 1; internal pull-up if DDRx = 0. | PORTD bit 4 = 1 → pull-up on PD4 |
| PINx | Present electrical logic level at the pin. | Read PIND bit 4 to sample PD4 |

With a button between an input and GND, an enabled pull-up holds the released input HIGH. Pressing the button connects it to GND, so the input reads LOW. This is an active-low input. Read PINx to observe it; PORTx holds the output or pull-up setting.

```
DDRD  &= ~(1 << PD4);  // PD4 input
PORTD |=  (1 << PD4);  // pull-up enabled
if ((PIND & (1 << PD4)) == 0) {
    // Input is LOW: the button is pressed.
}
```

## 2. Sixteen keys using eight AVR pins

A keypad switch connects one row wire to one column wire only while pressed. Four row wires and four column wires distinguish 16 intersections using eight pins. Rows are R0–R3 and columns are C0–C3 in our program; your TinkerCAD component may label them R1–R4 and C1–C4.

Every key represents a numeric value from 0 to 15. Four LEDs display that value in binary. The LED order is PB3 PB2 PB1 PB0 (D11 D10 D9 D8). A 1 means ON; a 0 means OFF.

| Row | C0 | C1 | C2 | C3 |
|---|---|---|---|---|
| R0 | 1 → 0001 | 2 → 0010 | 3 → 0011 | A → 1010 |
| R1 | 4 → 0100 | 5 → 0101 | 6 → 0110 | B → 1011 |
| R2 | 7 → 0111 | 8 → 1000 | 9 → 1001 | C → 1100 |
| R3 | * → 1110 | 0 → 0000 | # → 1111 | D → 1101 |

A, B, C, and D stand for values 10, 11, 12, and 13. In this exercise * is value 14 (1110) and # is value 15 (1111). They do not perform Clear, Cancel, Enter, or Confirm. Key 0 produces 0000, so all four LEDs turn OFF. With no key pressed, the LEDs keep the last accepted value; a fresh reset also starts with all LEDs OFF.

## 3. How one row is scanned

Set PD4–PD7 as column inputs with pull-ups. Test one row at a time: make the selected PC0–PC3 row an output that drives LOW, and leave the other rows as inputs with no pull-ups (high impedance). This prevents two row outputs from driving against each other if two keys are touched. Read PIND to find any LOW column, then restore the selected row to an input before trying the next row or returning.

For key 5, the physical switch joins R1 to C1. When R1 is the active LOW row, PD5 (C1) reads LOW. The upper four PIND bits read 1101 in order C3 C2 C1 C0. The program identifies row 1, column 1, looks up value 5, and displays 0101.

| Active row | Columns C3 C2 C1 C0 | Meaning if only key 5 is held |
|---|---|---|
| R0 | 1111 | No key on this row |
| R1 | 1101 | C1 LOW → key 5 |
| R2 | 1111 | No key on this row |
| R3 | 1111 | No key on this row |

The expression (PIND >> 4) & 0x0F moves PD4–PD7 into the lower four bits. A zero bit identifies the connected column because the inputs are active low. Only one key at a time is required in this activity.

## 4. Why the code waits for a stable key

Mechanical contacts can bounce when pressed or released. A fast CPU can see several HIGH/LOW changes during one physical action. Scan once, wait about 20 ms, then scan again; accept the key only if the same value is still present. After displaying it, wait for a stable release before accepting another press.

At F_CPU = 16 MHz, one clock period is 1/16,000,000 s = 62.5 ns; a two-cycle instruction takes about 125 ns. Twenty milliseconds spans about 320,000 clock periods. Not every AVR instruction takes one cycle, and a delay routine contains many instructions. The F_CPU setting must match the simulated CPU clock for _delay_ms() and _delay_us() to represent the intended time.

## 5. TinkerCAD circuit and program

Use the Arduino Uno in TinkerCAD as the simulation board for its ATmega328P. In the Code → Text editor, setup() and loop() are the editor's entry points. The I/O work must use AVR registers. Do not use pinMode(), digitalRead(), digitalWrite(), the Keypad library, or Arduino timing functions. A stand-alone bare-metal AVR C project would instead use main().

| Component connection | AVR pin | Uno label |
|---|---|---|
| Keypad R1, R2, R3, R4 | PC0, PC1, PC2, PC3 | A0, A1, A2, A3 |
| Keypad C1, C2, C3, C4 | PD4, PD5, PD6, PD7 | D4, D5, D6, D7 |
| LED bits 0, 1, 2, 3 | PB0, PB1, PB2, PB3 | D8, D9, D10, D11 |

Connect each LED from its output pin through a 220–330 Ω resistor to the LED anode; connect its cathode to GND. Check the keypad's printed row and column labels before wiring rather than assuming connector position. The keypad matrix itself needs no separate VCC or GND connection.

Program plan: configure the column inputs and pull-ups; leave rows inactive; scan one LOW row at a time; confirm the same key after 20 ms; display its four-bit code; wait for stable release. Preserve unrelated bits of PORTB, PORTC, and DDRC while changing only the pins used here.

## Starter code

Paste this into TinkerCAD's Text editor. Complete keypad_scan_once() yourself. Its return value must be -1 when no key is pressed or 0–15 for a key. Restore all four row pins to inactive inputs before every return.

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

/* Digits keep their values; A-D = 10-13, * = 14, # = 15. */
static const uint8_t key_code[4][4] = {
   { 1, 2, 3, 10 },   // 1 2 3 A
   { 4, 5, 6, 11 },   // 4 5 6 B
   { 7, 8, 9, 12 },   // 7 8 9 C
   {14, 0,15, 13 }    // * 0 # D
};

static int8_t keypad_scan_once(void)
{
   // TODO: For row = 0..3, leave all PC0..PC3 as inputs,
   // then make only the selected row a LOW output.
   // Wait 5 us; read PD4..PD7 from PIND. A LOW bit
   // identifies the column. Return key_code[row][col].
   // Make all rows inputs again before every return.
   return -1;
}

static void gpio_init(void)
{
   PORTC &= (uint8_t)~0x0F; // row input pull-ups off
   DDRC  &= (uint8_t)~0x0F; // rows initially inactive
   DDRD  &= (uint8_t)~0xF0; // PD4..PD7 inputs
   PORTD |= 0xF0;          // column pull-ups on
   DDRB  |= 0x0F;          // PB0..PB3 LED outputs
   PORTB &= (uint8_t)~0x0F; // LEDs initially off
}
```

```
static void keypad_task(void)
{
   int8_t first = keypad_scan_once(); // first reading
   if (first < 0) return;            // -1 = no key; 0 is valid
   _delay_ms(20);                    // allow bounce to settle
   if (keypad_scan_once() != first) return; // reject a changed reading

   PORTB = (PORTB & 0xF0) | ((uint8_t)first & 0x0F);

   // Wait until release is stable for 20 ms.
   do {
      while (keypad_scan_once() >= 0) { }
      _delay_ms(20);
   } while (keypad_scan_once() >= 0);
}

int main(void)
{
   gpio_init();
   while (1) {
      keypad_task();
   }
}
```

Implementation hint: keep PORTC bits 0–3 at 0 and select each row through DDRC. For row r, (DDRC & 0xF0) | (1 << r) makes only that row an output. Delay about 5 µs, then use (PIND >> 4) & 0x0F to inspect four column bits. Check one column at a time for 0. Restore DDRC bits 0–3 to 0 on every exit path. In Tinkercad, wire the Uno, 4 × 4 keypad, four LEDs, and four resistors as shown above. Open Code → Text, paste the whole starter program, complete the TODO, and start the simulation. Test one key at a time.

## Individual homework

Build the circuit, complete keypad_scan_once(), and run the simulation. Your program must identify one key at a time, reject a transient reading after the 20 ms press debounce, display the key code on PB3–PB0, and wait for stable release. Keep the * and # meanings shown in the table; do not turn them into commands.

### Checks to perform

1. Press 5. The LEDs PB3 PB2 PB1 PB0 should show 0101.
2. Press *. The LEDs should show 1110. This key represents value 14.
3. Press 0 after a nonzero key. The LEDs should all turn OFF. Release the key and confirm the display keeps its last value.
4. Press #. All four LEDs should turn ON (1111).

### Questions to answer

1. What does each of DDRx, PORTx, and PINx do? What changes in the meaning of PORTx when DDRx = 0?
2. Why can a 4 × 4 keypad distinguish 16 keys with only eight AVR pins?
3. Why do we check the key a second time after 20 ms?

### What to submit

Submit one forum reply per student containing your answers to the three questions, your complete source code, a screenshot of the wired circuit, and two screenshots showing the LED patterns for keys 5 and *. Explain in two or three sentences how your scan identifies a row and column. Attach your code file or paste the complete code in your reply.

Use this exercise to practice register-level AVR input and output. In the final project, a keypad is optional; a dependable way to read inputs and control outputs will still be useful.